Smart Workspace Manager

Complete Work Plan - Updated with SQL, Auth, Authorization, Azure SQL, GitHub Actions, and React DOM Basics

Version	2.0
Updated	26 July 2026
Environment	Python 3.12, venv, pip
Cloud strategy	Azure free-first for personal use and testing

Phase 1: 14 calendar days.
Phase 2: 6 active build days plus 1 contingency/finalization day.
Total planned duration: 21 calendar days.

Prepared as an independent Python learning project.

Contents

1. Work Plan Purpose and Assumptions

This work plan covers the complete development path from an empty project directory to the end of Phase 2. It now explicitly includes raw SQL querying, SQLAlchemy implementation, authentication, ownership-based authorization, Azure SQL Database creation, GitHub Actions CI/CD, and basic DOM/browser concepts through React.

Item	Assumption
Duration	21 calendar days: 14 days for Phase 1 and up to 7 days for Phase 2.
Daily effort	Flexible; a focused multi-hour development session is recommended.
Language/environment	Python 3.12, venv, pip, PowerShell, VS Code, Git, and GitHub.
Phase 1	Streamlit + SQLite + reusable Python services + raw SQL practice.
Phase 2	FastAPI + React/Vite + Azure SQL + authentication + authorization + GitHub Actions.
Cloud	Azure free plans for personal testing, subject to subscription eligibility.
Not required	Conda, Docker, Kubernetes, paid Azure plans, background queues, advanced ML, or advanced vanilla DOM work.

Schedule philosophy: Every day must end with a runnable state or a clearly isolated branch. Features are accepted only after a small test and a Git commit. Optional features are not allowed to delay the phase gate.

2. Definition of Done

2.1 Feature Definition of Done

- The feature works through the active UI or API.
- The core logic is located in backend/services/ or another correct backend layer.
- Expected and invalid inputs are handled.
- For database features, the raw SQL idea is understood before the SQLAlchemy implementation.
- For protected features, authentication and ownership behavior are tested.
- At least one automated test covers the reusable logic where practical.
- Configuration values and secrets are not hard-coded.
- README or relevant documentation is updated.
- A meaningful Git commit records the completed feature.

2.2 Phase Definition of Done

- All mandatory completion criteria pass.
- A clean venv can install dependencies and run the system.
- The cloud deployment passes a smoke test.
- Known limitations are documented.
- Optional unfinished work is moved to a future backlog, not left half-connected.

3. Overall Timeline

Period	Focus	Main outcome
Days 1-3	Foundation	Environment, structure, Python refresh, settings, logging, raw SQL basics, SQLite, storage, and database foundations.
Days 4-7	File management	Upload, validation, organization, metadata CRUD, raw SQL filters/aggregates, and dashboard/library.
Days 8-11	Data capabilities	CSV analysis, cleaning, charts, reports, and ML demonstration.
Days 12-14	Quality and Phase 1 release	Tests, refactor, docs, Azure Streamlit deployment, and gate review.
Days 15-17	Phase 2 backend, database, and auth	FastAPI, schemas/routes, Azure SQL, migrations, authentication, and authorization.

Period	Focus	Main outcome
Days 18-20	React, DOM basics, cloud, and CI/CD	React UI, DOM/browser concepts, API integration, GitHub Actions, deployment, and parity testing.
Day 21	Buffer/finalization	Fix defects, finalize documents, verify workflows, tag Phase 2, and run demonstration.

4. Phase 1 Work Plan - Days 1 to 14

Day 1 - Environment and repository

- Confirm Python 3.12 through the launcher.
- Activate the existing smart-workspace venv.
- Initialize Git and GitHub repository.
- Create the Phase 1 folder structure.
- Create .gitignore, .env.example, requirements files, and a minimal README.
- Install the initial packages and run a one-page Streamlit smoke test.

Daily output	A clean repository that starts through Streamlit.
Learning focus	venv, pip, imports, packages, modules, Git basics.

Day 2 - Python syntax refresh and configuration

- Implement backend/config/settings.py.
- Practice Python data types, collections, conditions, loops, functions, comprehensions, and exceptions while creating validators and constants.
- Create pathlib-based project path resolution.
- Configure logging.
- Add tests for simple validators and path helpers.

Daily output	Configuration, validation, and logging foundations.
Learning focus	Python syntax differences from Java, pathlib, functions, exceptions, type hints.

Day 3 - Database and storage foundations

- Write raw SQL CREATE TABLE examples before creating SQLAlchemy models.
- Write raw SQL INSERT, SELECT, UPDATE, and DELETE examples for the files table.
- Create SQLAlchemy engine/session configuration for SQLite.
- Define initial ORM models for files, reports, analysis jobs, logs, settings, and ML models.
- Create repository functions for file CRUD using SQLAlchemy.
- Compare raw SQL output with SQLAlchemy repository output.
- Create storage_service with local and Azure-path-compatible roots.
- Create a database initialization script and tests.
- Document the raw SQL query and its SQLAlchemy equivalent.

Daily output	SQLite database, raw SQL examples, SQLAlchemy repositories, and reusable local storage abstraction.
Learning focus	SQL basics, SQLite, SQLAlchemy models, sessions, CRUD, raw SQL vs ORM, configuration.

Day 4 - File upload service

- Implement safe file-name generation.
- Validate file extension and size.
- Save uploaded bytes through storage_service.

- Create metadata record and return a structured result.
- Handle duplicate names and partial failures.
- Add file-service unit and integration tests.

Daily output	Reliable upload workflow independent from Streamlit.
Learning focus	Binary files, pathlib, exceptions, dataclasses/objects, transactions.

Day 5 - Streamlit upload page

- Build the upload page and call file_service.
- Display success, validation, and error states.
- Show original/stored name, type, size, and location.
- Add upload limits to settings.
- Test several valid and invalid file types manually.

Daily output	Working browser-based file upload.
Learning focus	Streamlit widgets/state, UI-service separation, user feedback.

Day 6 - File organizer and automation logs

- Implement category detection and destination rules.
- Move or copy uploaded files safely into managed categories.
- Update database metadata after movement.
- Record automation logs.
- Add organizer tests for categories, duplicate destinations, and failures.

Daily output	Automatic organization with traceable logs.
Learning focus	Conditions, dictionaries, loops, file movement, custom exceptions, logging.

Day 7 - File library and dashboard

- Write raw SQL queries for file list, search, filter, count, and group-by-file-type dashboard summaries.
- Build repository filters and dashboard summary queries using SQLAlchemy.
- Compare raw SQL output with SQLAlchemy output.
- Create file-library page with search, type filter, download, and delete.
- Create dashboard cards and recent activity.
- Ensure deletion handles both storage and database records.
- Complete a Week 1 regression test and refactor duplicated code.

Daily output	Complete file-management module with SQL query understanding.
Learning focus	SELECT, WHERE, LIKE, ORDER BY, COUNT, GROUP BY, SQLAlchemy filtering and aggregation, reusable UI components.

Day 8 - CSV analysis service

- Load a selected CSV with safe limits.
- Return preview, shape, columns, data types, missing-value counts, duplicate count, and descriptive statistics.
- Handle encoding/parsing failures.
- Create structured analysis result objects.
- Write tests using small sample CSV files.

Daily output	Reusable pandas analysis service.
Learning focus	DataFrames, data types, exceptions, transformations, structured returns.

Day 9 - CSV analyzer page

- Create file selection and analyzer controls.
- Display preview and summary tables.
- Display missing values and duplicates.
- Allow column selection and basic filters.
- Save an analysis job record and status.

Daily output	Interactive data-analysis screen.
Learning focus	Streamlit data display, pandas integration, database workflow status.

Day 10 - Data cleaning and export

- Implement duplicate removal.
- Implement limited missing-value actions: drop rows, fill numeric values, fill text values.
- Keep the original file unchanged.
- Save the cleaned copy under processed/csv.
- Export CSV and Excel.
- Add cleaning tests.

Daily output	Controlled cleaning pipeline with downloadable outputs.
Learning focus	Copy vs mutation, functions, DataFrame operations, openpyxl, file output.

Day 11 - Charts and reports

- Create chart-selection logic based on column types.
- Implement at least bar, histogram, line, and scatter options where valid.
- Implement report_service for summary CSV/Excel/text or HTML reports.
- Save report metadata.
- Build the Reports page and report tests.

Daily output	Visual analysis and report generation.
Learning focus	Plotting, reusable functions, formatting, file generation, metadata.

Day 12 - Basic ML lab

- Use one small sample dataset with a clear target.
- Implement preprocessing suitable for the selected data.
- Split train/test data, train one simple model, calculate metrics, save with joblib, and predict one sample.
- Build the ML Demo page.
- Document that this is a learning feature, not an automated model for arbitrary datasets.

Daily output	One complete lightweight ML workflow.
Learning focus	scikit-learn pipeline, model lifecycle, metrics, persistence, limitations.

Day 13 - Testing, refactor, and release preparation

- Run all tests and add missing edge cases.
- Refactor Streamlit pages so they only call services.
- Add database/storage integration tests.
- Confirm raw SQL examples and SQLAlchemy equivalents are documented.
- Review logging and error messages.
- Freeze dependencies from the working environment.
- Complete README local setup and usage instructions.

Daily output	Stable Phase 1 release candidate.
Learning focus	pytest fixtures, test isolation, refactoring, dependency management, database documentation.

Day 14 - Azure Phase 1 deployment and gate

- Create one Azure resource group.
- Create an App Service F1 Linux web app using a supported Python runtime.
- Configure the Streamlit startup command and App Service settings.
- Use /home/data as the cloud data root.
- Deploy through GitHub or Deployment Center.
- Run the smoke test: open, upload, analyze, report, and restart check.
- Capture screenshots, document F1 limits, tag the release as phase-1.

Daily output	Phase 1 deployed and formally accepted.
Learning focus	Azure App Service, startup commands, environment variables, logs, quotas.

5. Phase 1 Gate Review

Gate item	Pass condition
Local run	A clean venv installation starts Streamlit.
Files	Upload, organize, list, filter, download, and delete pass.
SQL	Raw SQL and SQLAlchemy equivalents exist for main file/database operations.
Data	Analyze and clean a CSV without changing the original.
Charts	At least three valid chart types display.
Reports	At least one downloadable report format works.
ML	The documented sample workflow trains, evaluates, saves, and predicts.
Architecture	No major business logic remains inside Streamlit pages.
Tests	Mandatory test suite passes.
Azure	F1 deployment passes the smoke test.
Documentation	README, screenshots, SQL notes, limitations, and setup instructions are complete.

Stop/go rule: Do not start React until this gate passes. Fix architecture, SQL clarity, and service reuse first.

6. Phase 2 Work Plan - Days 15 to 21

The target is to complete the active build in six days. Day 21 is reserved for unresolved integration issues, documentation, workflow verification, and the final release.

Day 15 - FastAPI foundation

- Create backend/main.py.
- Add health route, CORS configuration, central exception handling, and dependency structure.
- Create initial Pydantic schemas.
- Wrap existing services with file and dashboard routes.
- Add protected-route structure even before full authentication is completed.
- Add API response pattern for 401 Unauthorized and 403 Forbidden.
- Run FastAPI locally with uvicorn.

- Add health and file API tests.

Daily output	Running FastAPI API that reuses Phase 1 services.
Learning focus	ASGI, routes, request/response models, decorators, status codes, protected route structure, dependency injection.

Day 16 - Complete API surface

- Add analysis, cleaning, reports, ML, and settings endpoints.
- Use consistent error response structures.
- Add file-download responses safely.
- Add ownership-aware API design to file, analysis, report, and ML endpoints.
- Generate or review OpenAPI documentation.
- Add API tests for valid, invalid, missing, unauthorized, forbidden, and failure cases.

Daily output	API parity for the main Phase 1 features with authorization-aware design.
Learning focus	REST design, HTTP methods, JSON, streaming/download responses, validation, authorization-aware endpoints.

Day 17 - Azure SQL, migrations, authentication, and authorization

- Create Azure SQL Database using the documented free offer if available in the subscription.
- Configure firewall/network access and database connection string.
- Configure SQLAlchemy for SQLite locally and Azure SQL in cloud.
- Initialize Alembic and create the first migration.
- Add users table and ownership fields such as user_id.
- Write raw SQL JOIN and ownership-filter examples.
- Implement password hashing.
- Implement login and token/session creation.
- Implement protected FastAPI dependencies.
- Implement ownership-based authorization: user can access only their own files, analysis jobs, reports, and ML models.
- Test local and Azure database connections.
- Test login, invalid login, missing token, invalid token, and forbidden access.
- Set the Azure SQL free-limit behavior to avoid billable continuation where available.
- Document a fallback if the Azure SQL free offer is unavailable for the subscription.

Daily output	Azure SQL, migrations, authentication, and basic ownership authorization working.
Learning focus	Azure SQL, raw SQL joins, SQLAlchemy relationships, Alembic migrations, password hashing, JWT/session logic, protected routes, 401 vs 403.

Day 18 - React/Vite frontend foundation

- Create the Vite React app inside frontend/.
- Add router, layout, navigation, API client, environment config, loading states, and error states.
- Build Login, Dashboard, File Upload, and File Library pages.
- Practice controlled inputs in login and upload forms.
- Practice button click events and form submit events.
- Practice conditional rendering for loading, success, validation error, and authentication states.
- Use useRef once for a simple direct DOM-related task such as focusing an input or resetting a file input.
- Connect to the local FastAPI backend.

Daily output	React app performing login and file-management API calls while demonstrating basic DOM/browser concepts.
Learning focus	React components, props, state, events, controlled inputs, conditional

	rendering, useRef, API calls, CORS.

Day 19 - React data features

- Build CSV Analyzer, Cleaning, Reports, and ML pages.
- Render returned tables and charts.
- Implement report/file downloads.
- Handle validation messages and token expiry.
- Add frontend handling for 401 Unauthorized and 403 Forbidden responses.
- Show user-friendly authorization error messages.
- Practice dynamic rendering of tables and charts from API responses.
- Run feature-parity checks against the Streamlit app.

Daily output	React covers all mandatory user workflows with auth-aware UI behavior.
Learning focus	Frontend data flow, auth-aware UI, conditional rendering, API error handling, dynamic UI rendering, chart integration.

Day 20 - Free Azure Phase 2 deployment and basic CI/CD

- Create backend GitHub Actions workflow.
- Backend workflow should install Python dependencies and run pytest.
- Deploy FastAPI to Azure App Service F1 after tests pass.
- Create frontend GitHub Actions workflow.
- Frontend workflow should install Node dependencies and build React.
- Deploy React to Azure Static Web Apps Free.
- Store Azure publish credentials, Static Web Apps token, API URL, database URL, and secrets safely using GitHub Secrets/Azure app settings.
- Configure FastAPI startup command, Azure SQL URL, secrets, CORS frontend URL, and /home/data root.
- Run cloud database migrations.
- Execute full cloud smoke tests.

Daily output	React, FastAPI, Azure SQL, and basic GitHub Actions CI/CD working on free Azure services.
Learning focus	GitHub Actions, CI/CD basics, GitHub Secrets, Azure deployment, frontend build, backend tests, cloud environment variables.

Day 21 - Contingency, legacy move, and final release

- Fix remaining defects and repeat smoke tests.
- Confirm GitHub Actions workflows run successfully from a fresh push.
- Move Streamlit files to frontend/legacy_streamlit/ after React parity is confirmed.
- Move Streamlit dependency to requirements-streamlit.txt.
- Document raw SQL examples and SQLAlchemy equivalents.
- Document authentication and authorization behavior.
- Document Azure SQL creation steps and any subscription limitations/fallbacks.
- Document CI/CD workflow files and GitHub Secrets used.
- Document React DOM/browser concepts practiced.
- Update README, API docs, Azure deployment notes, architecture screenshots, and known limitations.
- Run a clean local setup and final test suite.
- Tag the release as phase-2 and prepare the final demonstration.

Daily output	Completed and documented Phase 2 release.
Learning focus	Release management, GitHub Actions verification, backward reference preservation, documentation, final QA.

7. Phase 2 Gate Review

Gate item	Pass condition
React	The active frontend is hosted on Static Web Apps Free.
FastAPI	All mandatory workflows use documented API endpoints.
Reuse	Routes call existing service functions rather than copied logic.
SQL	Raw SQL examples and SQLAlchemy equivalents are documented.
Database	The deployed app reads and writes Azure SQL through migrations/models, or an eligibility fallback is documented.
Authentication	The personal user can log in and protected requests reject unauthenticated access.
Authorization	A user cannot access another user's files, reports, analyses, or models.
DOM/browser basics	React UI demonstrates controlled inputs, events, conditional rendering, dynamic rendering, and one useRef example.
CI/CD	Backend and frontend GitHub Actions workflows run successfully.
Files	Upload, list, download, and delete work in cloud testing.
Analysis	CSV analysis, cleaning, charts, reports, and ML demo work through React.
Testing	Unit, integration, API, auth, authorization, and CI checks pass.
Legacy	Streamlit remains under legacy_streamlit/ and can be run with its separate requirements.
Cost	Resources use intended free offers and budget/quota checks are documented.
Documentation	README, API, Azure, project, folder, CI/CD, SQL notes, and work-plan documents match the final system.

8. Daily Learning and Git Routine

Step	Daily practice
Start	Pull/confirm branch, activate venv, run tests, and read the day's target.
Learn	Study only the Python syntax/library concepts required for the task.
Implement	Write the smallest reusable backend function first, then connect the UI or API.
SQL step	For DB work, write or explain raw SQL first, then implement SQLAlchemy.
Security step	For protected work, add auth/authorization checks and tests.
Test	Run the relevant unit/integration/API tests and one manual happy-path test.
Refactor	Remove duplication, improve names/types, and confirm layer boundaries.
Document	Update README notes, environment variables, SQL notes, auth behavior, and known limitations.
Commit	Create one or more meaningful commits; avoid one giant end-of-day commit.
End	Run the application from the normal entry point and record the next task.

9. Testing Strategy by Milestone

Checkpoint	Required test focus
After Day 3	Settings, validators, database initialization, raw SQL examples, SQLAlchemy repository CRUD.
After Day 6	Upload, safe names, categories, movement, rollback/failure, and logs.
After Day 7	File filters, dashboard aggregates, raw SQL vs SQLAlchemy output checks.
After Day 8	CSV parsing, limits, missing values, duplicates, and statistics.
After Day 11	Cleaning outputs, chart preparation, reports, and metadata.
After Day 12	ML preprocessing, training, metrics, persistence, and prediction.

Checkpoint	Required test focus
After Day 14	Full Phase 1 regression and Azure smoke test.
After Day 16	API success/validation/not-found/unauthorized/forbidden/error tests.
After Day 17	Auth, Azure SQL connection, migration, ownership, raw SQL JOIN/ownership query tests.
After Day 19	React/API parity, auth-aware UI, 401/403 handling, DOM/browser behavior manual test.
After Day 20	GitHub Actions backend/frontend workflows and cloud end-to-end smoke test.
Day 21	Fresh setup, full test suite, workflow verification, and release acceptance.

10. Azure Deployment and CI/CD Checklist

10.1 Phase 1 App Service F1

- Resource group created.
- Linux App Service F1 selected.
- Python runtime compatible with project dependencies selected.
- requirements.txt detected and installed.
- Streamlit startup command configured.
- DATA_ROOT=/home/data configured.
- SECRET_KEY and environment variables configured in App Service settings.
- Log stream checked.
- Upload, analysis, report, and restart smoke tests passed.
- F1 quotas and no-production-SLA limitation documented.

10.2 Phase 2 Static Web Apps + FastAPI + Azure SQL + GitHub Actions

- React repository/build settings correct for frontend/.
- Static Web Apps Free selected.
- Frontend API base URL configured.
- App Service repurposed with FastAPI startup command.
- CORS allows only the local and deployed frontend origins.
- Azure SQL free offer selected if available; free-limit behavior configured safely.
- DATABASE_URL, auth secrets, API URL, Azure publish credentials, and Static Web Apps token stored safely.
- Backend GitHub Actions workflow installs Python dependencies and runs pytest.
- Backend workflow deploys FastAPI to Azure App Service after tests pass.
- Frontend GitHub Actions workflow installs Node dependencies and builds React.
- Frontend workflow deploys React to Azure Static Web Apps.
- Migrations executed.
- Health, login, ownership/forbidden check, upload, analysis, report, and delete smoke tests passed.
- Azure Cost Management/budget alerts checked.

11. Deliverables Matrix

Deliverable group	Target days	Evidence
Project setup	D1-D3	Repository, venv, folders, settings, logging, raw SQL examples, SQLite database.
File management	D4-D7	Upload, organizer, library, dashboard, metadata, raw SQL filters/aggregates, tests.
Data analysis	D8-D10	Analyzer, cleaning, exports, sample data tests.
Visualization/reporting	D11	Charts, reports, report history.
ML	D12	One documented basic ML workflow.
Phase 1 release	D13-D14	Tests, README, Azure Streamlit deployment, tag.
FastAPI	D15-D16	API, schemas, docs, protected-route structure, tests.

Deliverable group	Target days	Evidence
Database/auth/authz	D17	Azure SQL, Alembic, login/protected routes, ownership authorization.
React + DOM basics	D18-D19	Active professional frontend with parity, controlled inputs, events, conditional rendering, useRef.
Phase 2 cloud + CI/CD	D20	Static Web Apps + App Service + Azure SQL + GitHub Actions workflows.
Final release	D21	Legacy move, SQL notes, auth/authz docs, CI/CD docs, QA, tag, demo.

12. Contingency and Scope-Control Plan

Priority	Scope
Mandatory	Upload, organize, metadata, library, raw SQL basics, SQLAlchemy repositories, CSV summary, basic cleaning, reports, tests, Streamlit deployment, FastAPI, React, Azure SQL attempt, authentication, ownership authorization, basic GitHub Actions.
Should have	Multiple charts, Excel export, ML persistence, frontend auth polish, automatic deployment from main branch.
Optional	PDF report, advanced cleaning UI, complex model selection, roles, password reset, background jobs, Blob Storage, Docker.
Remove first when delayed	PDF, extra chart types, arbitrary-dataset ML, frontend animations, advanced settings, advanced CI/CD polish.
Never compromise	Service separation, raw SQL understanding, safe file handling, secrets, database correctness, auth/authorization correctness, core tests, documentation accuracy.

Phase 2 under seven days: The schedule is achievable only because Phase 2 reuses a tested backend service layer, keeps authorization ownership-based only, and implements basic GitHub Actions rather than advanced release engineering.

13. Final Demonstration Script

1. Open the deployed React application and log in.
2. Show that a protected route rejects unauthenticated access.
3. Show the dashboard and explain that React calls FastAPI.
4. Upload a small CSV and show its stored metadata.
5. Explain ownership-based authorization and show that file queries are user-scoped.
6. Show one raw SQL query and the equivalent SQLAlchemy repository code.
7. Open the file library and demonstrate filtering.
8. Analyze the CSV and show preview, data types, missing values, duplicates, and summary statistics.
9. Apply one cleaning operation and download the cleaned file.
10. Generate and download a report.
11. Run the small ML demonstration and show metrics/prediction.
12. Open the FastAPI documentation and show one endpoint contract.
13. Show Azure architecture: Static Web Apps Free, App Service F1, and Azure SQL or documented fallback.
14. Show Azure SQL Database in the Azure portal if the free offer was available.
15. Show the GitHub Actions workflow run for backend tests/deployment.
16. Show the GitHub Actions workflow run for frontend build/deployment.
17. Show one React form using controlled inputs, event handling, conditional rendering, and one simple useRef example.
18. Show the repository structure and explain service reuse and legacy_streamlit/.
19. Show the passing test result and state the free-tier limitations and future publication upgrade path.